



南開大學
Nankai University

计算机学院
数据安全实验报告

零知识证明实践

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 5 月 5 日

目录

1 实验要求	2
2 实验原理	2
2.1 零知识证明基本思想	2
2.2 Schnorr 协议与 Fiat-Shamir 变换	2
2.3 将三次方程转化为曲线点关系	3
2.4 实现方案说明	3
3 实验过程	3
3.1 导入库并设置公开参数	3
3.2 实现椭圆曲线基础运算	3
3.3 实现哈希编码和 Fiat-Shamir 挑战	5
3.4 实现 Chaum-Pedersen 子证明	6
3.5 实现 Alice 的证明生成算法	7
3.6 实现验证者的证明验证算法	8
3.7 运行正确样例	9
3.8 篡改测试与错误见证测试	10
4 实验结果分析	11
4.1 正确性分析	11
4.2 可靠性分析	12
4.3 零知识性分析	12
5 结论与实践心得	12

1 实验要求

参考教材第四章零知识证明部分，完成一个证明生成与证明验证实验。实验设定为：Alice 希望证明自己知道方程

$$x^3 + x + 5 = out$$

的解，其中 out 是公开值。本实验设定 $out = 35$ ，Alice 的秘密见证为 $x = 3$ ，因为

$$3^3 + 3 + 5 = 27 + 3 + 5 = 35.$$

实验需要实现如下内容：

1. 用代码实现证明生成过程，使 Alice 可以基于秘密见证 x 生成证明；
2. 用代码实现证明验证过程，使验证者在不知道 x 的情况下验证证明是否正确；
3. 在报告中说明实验原理、实验流程、关键代码、运行结果与正确性分析。

本实验最终实现文件为 `zk_polynomial_proof_lab.ipynb`。

2 实验原理

2.1 零知识证明基本思想

零知识证明允许证明者向验证者证明某个命题为真，同时不泄露除“命题为真”之外的额外信息。对本实验而言，公开命题可以写为：

$$C(x) : x^3 + x + 5 = 35.$$

Alice 要证明自己知道一个满足 $C(x)$ 的秘密值 x ，但不把 x 直接发给验证者。

零知识证明通常需要满足三个性质：

1. 完备性：如果 Alice 确实知道正确见证，诚实生成的证明应通过验证；
2. 可靠性：如果 Alice 不知道正确见证，则很难伪造一个能通过验证的证明；
3. 零知识性：验证者只知道 Alice 知道某个满足条件的见证，而不知道见证本身。

2.2 Schnorr 协议与 Fiat-Shamir 变换

教材中介绍的 Schnorr 协议可以证明“证明者知道某个离散对数”。设椭圆曲线基点为 G ，秘密值为 a ，公开点为 $PK = aG$ 。证明者可以在不公开 a 的情况下证明自己知道 PK 对应的秘密值。

交互式 Schnorr 协议需要验证者给出随机挑战。为了减少交互，可以使用 Fiat-Shamir 变换：将公开语句、承诺点等信息输入哈希函数，用哈希输出代替验证者的随机挑战。这样证明者只需发送一轮证明，验证者即可独立验证。

2.3 将三次方程转化为曲线点关系

本实验使用椭圆曲线群来表示多项式关系。设 G 是公开基点，Alice 根据秘密 x 计算：

$$A = xG, \quad B = xA = x^2G, \quad C = xB = x^3G.$$

若这些点确实由同一个 x 生成，则有：

$$C + A + 5G = (x^3 + x + 5)G = outG.$$

因此验证者可以检查曲线点等式 $C + A + 5G = outG$ 。

但是，仅检查该等式还不够，因为验证者还需要确认 A, B, C 之间确实使用了同一个秘密 x 。本实验使用两个 Chaum-Pedersen 子证明：

$$\log_G(A) = \log_A(B),$$

用于证明 $A = xG$ 且 $B = xA$ ；

$$\log_G(A) = \log_B(C),$$

用于证明 $A = xG$ 且 $C = xB$ 。两个证明共同约束 A, B, C 是由同一个秘密 x 逐步生成的。

2.4 实现方案说明

实验代码直接用 Python 实现椭圆曲线基础运算、Fiat-Shamir 哈希挑战、Chaum-Pedersen 证明生成与验证。该实现不依赖第三方密码库，便于观察每一步计算过程。

3 实验过程

3.1 导入库并设置公开参数

首先导入哈希函数、随机数和格式化输出模块，然后设置曲线参数。该曲线满足 $y^2 = x^3 + 7 \pmod{P}$ ，其中 G 是公开基点， N 是基点阶。

这些参数都是公开参数，不属于 Alice 的秘密。验证者也使用同样参数完成验证。该代码片段只做初始化，因此运行后无直接输出。

3.2 实现椭圆曲线基础运算

教材中使用 aG 表示秘密数 a 与基点 G 的标量乘法。本实验中的 `scalar_mul(a, G)` 就是在计算该值。下面代码实现模逆、曲线点合法性检查、点取负、点加法和标量乘法。

Listing 1: 椭圆曲线点运算实现

```

1 def mod_inv(value, modulus):
2     """计算 value 在模 modulus 下的乘法逆元。"""
3     return pow(value % modulus, -1, modulus)
4
5 def is_on_curve(point):
6     """检查点是否在 secp256k1 曲线上。"""

```

```

7     if point is INF:
8         return True
9     x, y = point
10    return (y * y - (x * x * x + A_CURVE * x + B_CURVE)) % P == 0
11
12 def point_neg(point):
13     """计算椭圆曲线点的相反数。"""
14     if point is INF:
15         return INF
16     x, y = point
17     return (x, (-y) % P)
18
19 def point_add(left, right):
20     """椭圆曲线点加法。"""
21     if left is INF:
22         return right
23     if right is INF:
24         return left
25
26     x1, y1 = left
27     x2, y2 = right
28
29     # 如果两个点互为相反数，相加得到无穷远点。
30     if x1 == x2 and (y1 + y2) % P == 0:
31         return INF
32
33     if left == right:
34         # 点加倍时的斜率。
35         slope = (3 * x1 * x1 + A_CURVE) * mod_inv(2 * y1, P) % P
36     else:
37         # 普通点加法时的斜率。
38         slope = (y2 - y1) * mod_inv(x2 - x1, P) % P
39
40     x3 = (slope * slope - x1 - x2) % P
41     y3 = (slope * (x1 - x3) - y1) % P
42     return (x3, y3)
43
44 def scalar_mul(k, point):
45     """椭圆曲线标量乘法：计算 k * point。"""
46     if point is INF or k % N == 0:
47         return INF
48     if k < 0:
49         return scalar_mul(-k, point_neg(point))
50
51     k = k % N
52     result = INF
53     addend = point
54
55     # 二进制 double-and-add 算法。

```

```

56 while k:
57     if k & 1:
58         result = point_add(result, addend)
59         addend = point_add(addend, addend)
60         k >>= 1
61     return result
62
63 # 基本自检: 基点应在曲线上, 并且  $N \cdot G$  是无穷远点。
64 assert is_on_curve(G)
65 assert scalar_mul(N, G) is INF
66 print("椭圆曲线基础运算自检通过。")
67

```

运行结果:

Listing 2: 椭圆曲线基础运算自检输出

```

1 椭圆曲线基础运算自检通过。
2

```

结果分析: 基点 G 位于曲线上, 并且 $N \cdot G$ 得到无穷远点, 说明曲线参数与点运算实现一致。如果这里断言失败, 则后续证明中的点关系验证没有意义。

3.3 实现哈希编码和 Fiat-Shamir 挑战

Fiat-Shamir 变换需要用哈希函数生成挑战 c 。哈希输入必须包含公开语句、基点、结果点和承诺点, 并且编码时需要区分整数、字符串和曲线点, 避免不同数据拼接后产生歧义。

Listing 3: 哈希编码与 Fiat-Shamir 挑战

```

1 def int_to_32_bytes(value):
2     """把整数编码成 32 字节, 用于哈希输入。"""
3     return int(value % N).to_bytes(32, "big")
4
5 def encode_point(point):
6     """把椭圆曲线点编码成字节串。"""
7     if point is INF:
8         return b"\x00"
9     x, y = point
10    return b"\x04" + x.to_bytes(32, "big") + y.to_bytes(32, "big")
11
12 def encode_item(item):
13    """给哈希函数准备无歧义编码, 防止不同类型数据拼接后产生混淆。"""
14    if item is INF or isinstance(item, tuple):
15        data = b"P" + encode_point(item)
16    elif isinstance(item, int):
17        data = b"I" + int_to_32_bytes(item)
18    elif isinstance(item, str):
19        data = b"S" + item.encode("utf-8")
20    else:

```

```

21         raise TypeError(f"不支持的哈希输入类型: {type(item)}")
22     return len(data).to_bytes(4, "big") + data
23
24 def hash_challenge(label, *items):
25     """Fiat-Shamir 变换: 用 SHA-256 生成不可预测挑战 c。"""
26     digest = hashlib.sha256()
27     digest.update(encode_item(label))
28     for item in items:
29         digest.update(encode_item(item))
30     return int.from_bytes(digest.digest(), "big") % N
31
32 def point_to_hex(point):
33     """把点转成便于查看的十六进制形式。"""
34     if point is INF:
35         return "INF"
36     x, y = point
37     return {"x": hex(x), "y": hex(y)}
38

```

运行结果: 该片段定义辅助函数, 无直接输出。

结果分析: 证明者和验证者会使用完全相同的公开输入调用 `hash_challenge`, 从而得到相同挑战 c 。如果证明被篡改, 哈希输入随之变化, 验证等式将无法通过。

3.4 实现 Chaum-Pedersen 子证明

本实验需要证明两个离散对数相等。证明目标为存在同一个 `witness`, 使得:

$$\text{result1} = \text{witness} \cdot \text{base1}, \quad \text{result2} = \text{witness} \cdot \text{base2}.$$

证明者随机选择 r , 计算 $t_1 = r \cdot \text{base1}$, $t_2 = r \cdot \text{base2}$, 再计算挑战 c 和响应 $z = r + c \cdot \text{witness}$ 。

Listing 4: Chaum-Pedersen 证明生成与验证

```

1 def prove_equal_logs(base1, result1, base2, result2, witness, label):
2     """
3     生成非交互式 Chaum-Pedersen 证明。
4     证明目标: result1 = witness * base1 且 result2 = witness * base2。
5     """
6     # 1. 随机选择盲因子 r, 并计算两个承诺点。
7     r = secrets.randbelow(N - 1) + 1
8     t1 = scalar_mul(r, base1)
9     t2 = scalar_mul(r, base2)
10
11     # 2. 用哈希生成挑战 c。
12     c = hash_challenge(label, base1, result1, base2, result2, t1, t2)
13
14     # 3. 计算响应 z = r + c * witness。
15     z = (r + c * witness) % N
16     return {"t1": t1, "t2": t2, "z": z}

```

```

17
18 def verify_equal_logs(base1, result1, base2, result2, proof, label):
19     """验证非交互式 Chaum-Pedersen 证明。"""
20     t1 = proof["t1"]
21     t2 = proof["t2"]
22     z = proof["z"]
23
24     # 验证所有点都在曲线上，避免无效点攻击。
25     points = [base1, result1, base2, result2, t1, t2]
26     if not all(is_on_curve(point) for point in points):
27         return False
28
29     # 验证者用同样的公开输入重新计算挑战 c。
30     c = hash_challenge(label, base1, result1, base2, result2, t1, t2)
31
32     # 检查 Schnorr 型等式:  $z \cdot \text{base} = t + c \cdot \text{result}$ 。
33     check1 = scalar_mul(z, base1) == point_add(t1, scalar_mul(c, result1))
34     check2 = scalar_mul(z, base2) == point_add(t2, scalar_mul(c, result2))
35     return check1 and check2
36

```

运行结果：该片段定义证明函数，无直接输出。

结果分析：若证明者知道 `witness`，则

$$z \cdot \text{base1} = (r + c \cdot \text{witness})\text{base1} = t_1 + c \cdot \text{result1}.$$

第二个等式同理。因此验证者可以确认两个离散对数相等，但不会直接得到 `witness`。

3.5 实现 Alice 的证明生成算法

Alice 先检查 x 是否满足整数方程，然后计算 $A = xG, B = xA, C = xB$ ，最后生成两个子证明。

Listing 5: Alice 生成三次方程零知识证明

```

1 def generate_polynomial_proof(x, out):
2     """
3     Alice 生成证明。
4     私密见证: x
5     公开声明: out
6     目标: 证明  $x^3 + x + 5 = \text{out}$ ，但不直接公开 x。
7     """
8     # 实验中先在整数范围检查见证是否满足题目方程。
9     if x ** 3 + x + 5 != out:
10         raise ValueError("见证 x 不满足  $x^3 + x + 5 = \text{out}$ ，不能生成诚实证明。")
11
12     # 把 x、x^2、x^3 映射到椭圆曲线点。
13     A_point = scalar_mul(x, G)      # A = xG
14     B_point = scalar_mul(x, A_point) # B = xA = x^2G
15     C_point = scalar_mul(x, B_point) # C = xB = x^3G

```

```

16
17 # 证明 A 和 B 使用同一个 x:  $\log_G(A) = \log_A(B)$ 。
18 cp1 = prove_equal_logs(
19     G, A_point, A_point, B_point, x, label=f"cp1|out={out}"
20 )
21
22 # 证明 A 和 C 使用同一个 x:  $\log_G(A) = \log_B(C)$ 。
23 cp2 = prove_equal_logs(
24     G, A_point, B_point, C_point, x, label=f"cp2|out={out}"
25 )
26
27 return {
28     "out": out,
29     "A": A_point,
30     "B": B_point,
31     "C": C_point,
32     "cp1": cp1,
33     "cp2": cp2,
34 }
35

```

运行结果：该片段定义证明生成函数，无直接输出。

代码解释：cp1 证明 $B = xA = x^2G$ ，cp2 证明 $C = xB = x^3G$ 。返回的证明对象包含 out、A、B、C、cp1、cp2，但不包含明文 x。

3.6 实现验证者的证明验证算法

验证者只使用证明对象完成验证，不需要 Alice 的秘密见证。

Listing 6: 验证者验证三次方程零知识证明

```

1 def verify_polynomial_proof(proof):
2     """验证者验证 Alice 发来的证明。验证过程中不需要知道 x。"""
3     out = proof["out"]
4     A_point = proof["A"]
5     B_point = proof["B"]
6     C_point = proof["C"]
7
8     # 1. 检查公开证明中的点是否合法。
9     if not all(is_on_curve(point) for point in [A_point, B_point, C_point]):
10         return False
11     if A_point is INF or B_point is INF or C_point is INF:
12         return False
13
14     # 2. 检查两个 Chaum-Pedersen 证明，确认 A、B、C 由同一个 x 关联生成。
15     cp1_ok = verify_equal_logs(
16         G, A_point, A_point, B_point, proof["cp1"], label=f"cp1|out={out}"
17     )
18

```

```

19 cp2_ok = verify_equal_logs(
20     G, A_point, B_point, C_point, proof["cp2"], label=f"cp2|out={out}")
21 )
22
23 # 3. 检查多项式输出:  $C + A + 5G$  是否等于  $outG$ 。
24 left = point_add(point_add(C_point, A_point), scalar_mul(5, G))
25 right = scalar_mul(out, G)
26 equation_ok = left == right
27
28 return cp1_ok and cp2_ok and equation_ok
29

```

运行结果：该片段定义验证函数，无直接输出。

结果分析：若两个子证明均通过，则验证者相信 $A = xG, B = x^2G, C = x^3G$ 。再检查 $C + A + 5G = outG$ ，即可验证 $x^3 + x + 5 = out$ 。本实验数值很小，远小于群阶 N ，因此曲线群上的等式与普通整数方程一致。

3.7 运行正确样例

设置 $x = 3, out = 35$ ，生成证明并验证。

Listing 7: 生成证明并验证

```

1 def printable_proof(proof):
2     """把证明转成便于阅读的形式。实际协议中会把这些字段序列化发送给验证者。"""
3     return {
4         "out": proof["out"],
5         "A": point_to_hex(proof["A"]),
6         "B": point_to_hex(proof["B"]),
7         "C": point_to_hex(proof["C"]),
8         "cp1": {
9             "t1": point_to_hex(proof["cp1"]["t1"]),
10            "t2": point_to_hex(proof["cp1"]["t2"]),
11            "z": hex(proof["cp1"]["z"]),
12        },
13        "cp2": {
14            "t1": point_to_hex(proof["cp2"]["t1"]),
15            "t2": point_to_hex(proof["cp2"]["t2"]),
16            "z": hex(proof["cp2"]["z"]),
17        },
18    }
19
20 # Alice 的秘密见证和公开输出。
21 x = 3
22 out = 35
23
24 # Alice 生成证明，并发送给验证者。
25 proof = generate_polynomial_proof(x, out)
26

```

```

27 print("生成的证明中没有直接出现 x =", x)
28 print("证明字段如下: ")
29 pprint(printable_proof(proof), sort_dicts=False)
30
31 # 验证者验证证明。
32 result = verify_polynomial_proof(proof)
33 print("\n验证结果: ", result)
34

```

一次运行得到的主要输出如下。由于证明中使用随机盲因子，每次运行时 cp1 和 cp2 的承诺点、响应值可能不同，但最终验证结果应始终为 True。

Listing 8: 正确证明运行结果

```

1 生成的证明中没有直接出现 x = 3
2 证明字段如下:
3 {'out': 35,
4  'A': {'x': '0xf9308a019258c31049344f85f89d5229b531c845836f99b08601f113bce036f9',
5        'y': '0x388f7b0f632de8140fe337e62a37f3566500a99934c2231b6cb9fd7584b8e672'},
6  'B': {'x': '0xacd484e2f0c7f65309ad178a9f559abde09796974c57e714c35f110dfc27ccbe',
7        'y': '0xcc338921b0a7d9fd64380971763b61e9add888a4375f8e0f05cc262ac64f9c37'},
8  'C': {'x': '0xdaed4f2be3a8bf278e70132fb0beb7522f570e144bf615c07e996d443dee8729',
9        'y': '0xa69dce4a7d6c98e8d4a1aca87ef8d7003f83c230f3afa726ab40e52290be1c55'},
10 'cp1': {'t1': {'x': '0x79be2931c2500667954266a3f4190003046723e375a30462d5f687010c43cc43',
11               'y': '0xeeb3b236a589af978c794445903212270408806e42e25aaf49ab7b5e7b24272d'},
12         't2': {'x': '0x45123b0cb25cbc62d9d4a83ae262f627a32248e38d0e50ec1912a7ae2d0d5d',
13               'y': '0x503b51cb3590cb441efd45d4f9395c6606c3e70ec5f1207eeebf7f1d0b9f6636'},
14         'z': '0x13af1d738bb1fa7e6473b8eef917ad8e8cd2d851b97c6d84ead1d4e5a9fe6e19'},
15 'cp2': {'t1': {'x': '0xd1622ab5b9a8401c5cf1793d02d48119b68f5592fb5f5d9c74da7d1bc1245b45',
16               'y': '0xba615b1964305a31cfc5c8ba36cf670e525f0c6ec03a844a16b265db7fd9cbe'},
17         't2': {'x': '0x1972c280ebab9b99725ac67ae1467fb074fbac77a9cca29d2e1909d0cdb6c8f4',
18               'y': '0x427c3f5f629a4419746e6272c20c8a48709a0791416f3e7986d6f37949187741'},
19         'z': '0x922a7320e42ee0ea0b133d86a345c5e4776afbd43a4afc00e93d24425077582'}}
20
21 验证结果: True
22

```

结果分析:

1. 输出的证明字段中没有明文 x 字段，说明验证者没有直接获得 Alice 的秘密见证；
2. A,B,C 分别对应 xG, x^2G, x^3G ；
3. cp1,cp2 证明这些点由同一个秘密 x 关联生成；
4. 验证结果: True 说明正确见证 $x = 3$ 可以通过验证。

3.8 篡改测试与错误见证测试

为了验证程序不会无条件接受证明，本实验继续构造失败样例：篡改公开值 out、篡改响应 z、使用错误见证 $x = 2$ 。

Listing 9: 篡改证明和错误见证测试

```

1 # 篡改测试 1: 把公开 out 从 35 改成 36。
2 tampered_out_proof = dict(proof)
3 tampered_out_proof["out"] = 36
4 print("篡改 out 后的验证结果: ", verify_polynomial_proof(tampered_out_proof))
5
6 # 篡改测试 2: 改动证明中的一个响应 z。
7 tampered_z_proof = dict(proof)
8 tampered_z_proof["cp1"] = dict(proof["cp1"])
9 tampered_z_proof["cp1"]["z"] = (tampered_z_proof["cp1"]["z"] + 1) % N
10 print("篡改 z 后的验证结果: ", verify_polynomial_proof(tampered_z_proof))
11
12 # 错误见证测试: x = 2 不满足 2^3 + 2 + 5 = 35, 诚实证明生成函数会拒绝。
13 try:
14     wrong_proof = generate_polynomial_proof(2, 35)
15 except ValueError as error:
16     print("错误见证生成证明失败: ", error)
17

```

运行结果:

Listing 10: 篡改测试运行结果

```

1 篡改 out 后的验证结果: False
2 篡改 z 后的验证结果: False
3 错误见证生成证明失败: 见证 x 不满足 x^3 + x + 5 = out, 不能生成诚实证明。
4

```

结果分析:

1. 将 out 从 35 改成 36 后, $C + A + 5G = 36G$ 不成立, 因此验证失败;
2. 将 z 加 1 后, Chaum-Pedersen 验证等式被破坏, 因此验证失败;
3. $x = 2$ 时 $2^3 + 2 + 5 = 15$, 不满足 $out = 35$, 因此证明生成函数拒绝生成诚实证明。

4 实验结果分析

4.1 正确性分析

正确样例中, Alice 使用 $x = 3$, 因此:

$$A = 3G, \quad B = 3A = 9G, \quad C = 3B = 27G.$$

验证者检查:

$$C + A + 5G = 27G + 3G + 5G = 35G = outG.$$

因此验证结果为 **True**。这体现了零知识证明的完备性。

4.2 可靠性分析

如果证明者不知道满足方程的 x ，就需要同时伪造两个 Chaum-Pedersen 子证明，并让最终曲线点方程成立。由于挑战值 c 由 SHA-256 根据公开输入和承诺点生成，证明者不能在不知道见证的情况下自由控制挑战。篡改 out 或 z 后验证失败，说明证明与公开声明和响应值之间存在严格绑定关系。

4.3 零知识性分析

证明对象中包含的是曲线点 A, B, C 和子证明 $cp1, cp2$ ，没有直接包含 x 。验证者通过离散对数关系和曲线点等式进行验证，而不是读取明文 x 。

需要注意的是，本实验中的 $x = 3$ 很小，方程也很简单，实际中验证者可以通过穷举猜出 x 。因此本实验主要用于展示协议机制，而不是保护小范围秘密。真实场景应使用更大的秘密空间、更完整的电路约束和经过审计的密码实现。

5 结论与实践心得

本实验基于教材中零知识证明、Schnorr 协议和 Fiat-Shamir 变换的思想，实现了一个用于证明 $x^3 + x + 5 = out$ 的非交互式零知识证明程序。

通过实验可以得到以下结论：

1. 当 Alice 使用正确见证 $x = 3$ 、公开值 $out = 35$ 时，证明可以成功生成并通过验证；
2. 验证者不需要知道 x ，只需要验证曲线点关系和两个 Chaum-Pedersen 子证明；
3. 篡改公开输出或证明响应都会导致验证失败，说明证明与公开声明之间存在绑定关系；
4. 错误见证 $x = 2$ 不能生成诚实证明，说明实验逻辑能够区分正确见证和错误见证。

本次实验加深了我对零知识证明流程的理解。教材中 Schnorr 协议的核心是“知道离散对数但不公开离散对数”，而本实验进一步将三次方程拆成多个椭圆曲线点关系，并用 Chaum-Pedersen 证明约束同一个秘密值在不同乘法步骤中的一致性。通过这种方式，可以把普通代数方程转化为可验证的密码学关系。

同时，本实验也说明零知识证明工程实现需要非常谨慎。代码只是简单实现，但真实应用还需要考虑安全参数、随机数质量、输入编码、无效点检查、证明系统选择和代码审计等问题。